ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	**CIENCIAS DE LA COMPUTACIÓN**	**CARRERA:**	Tecnologías de la Información		
ASIGNATURA:	Metodologías de Desarrollo de Software	**NIVEL:**	3	**FECHA:**	20/07/2026
DOCENTE:	Ing. Kevin Chuquitarco, Mgtr.	**PRÁCTICA N°:**	5	**CALIFICACIÓN:**	

Creación de Diagramas con PlantUML

Iker Mathews Guallo Vera, Jeffry David Vera Borja

RESUMEN

La presente práctica de laboratorio abordó el modelado del sistema NovaMarket, una plataforma de gestión comercial orientada a micronegocios desarrollada previamente por el equipo, mediante la elaboración de diagramas UML utilizando la herramienta PlantUML. Se configuró un entorno de trabajo en Visual Studio Code con la extensión PlantUML, apoyado en Java y Graphviz como motores de generación gráfica, y se desarrollaron cinco diagramas correspondientes a distintas vistas del sistema: casos de uso, clases, secuencia, componentes y despliegue.

El diagrama de casos de uso permitió representar a los actores Usuario, Administrador y Cajero identificados en las historias de usuario del Product Backlog, junto con sus interacciones principales sobre los módulos de micronegocios, productos, ventas y reportes. El diagrama de clases estructuró las entidades Usuario, Administrador, Cajero, Micronegocio, Producto, Venta y Reporte, definiendo sus atributos, métodos y relaciones de herencia, asociación y agregación. El diagrama de secuencia detalló el flujo temporal del proceso de registro de una venta y su efecto sobre el inventario, mientras que los diagramas de componentes y despliegue representaron la organización modular del sistema y su infraestructura de ejecución basada en un servidor de aplicación y un servidor de base de datos.

Los resultados evidencian que PlantUML constituye una alternativa eficiente frente a las herramientas gráficas tradicionales de modelado, dado que permite generar diagramas reproducibles y versionables a partir de texto plano, reduciendo el tiempo de elaboración y facilitando el mantenimiento de la documentación técnica del proyecto NovaMarket.

Palabras clave: UML, PlantUML, diagramas de software, modelado de sistemas, ingeniería de software.

1. INTRODUCCIÓN:

En el desarrollo de software se requiere de varias herramientas las cuales permitan representar, analizar y comunicar la estructura y el comportamiento de un sistema, antes y durante su implementación. El Lenguaje Unificado de Modelado, conocido como UML, se ha convertido en el más utilizado para este propósito, ya que su objetivo es representar gráficamente la estructura y el comportamiento de un sistema antes de su implementación. (Booch, Rumbaugh y Jacobson, 2005).

Tradicionalmente, los diagramas UML se elaboraban mediante editores gráficos de arrastrar y soltar, lo que dificultaba su versionado y generaba inconsistencias cuando varias personas trabajaban sobre el mismo modelo. Frente a esta limitación surgieron herramientas de modelado basadas en texto, entre las cuales PlantUML se ha posicionado como una de las más adoptadas por la comunidad de desarrollo, ya que permite describir un diagrama mediante una sintaxis simple y generar automáticamente su representación gráfica (PlantUML, s.f.).

En la presente práctica se aplicó PlantUML al desarrollo de NovaMarket, un sistema de gestión comercial para micronegocios cuyo Product Backlog fue construido previamente por el equipo en el laboratorio de artefactos Scrum del curso de Metodologías de Desarrollo de Software. A partir de las épicas y las historias de usuario definidas para dicho sistema, entre ellas la gestión de micronegocios, la gestión de productos, la gestión de ventas y la gestión de reportes, se elaboraron los diagramas de casos de uso, clases, secuencia, componentes y despliegue, con el fin de documentar de manera formal su arquitectura y su comportamiento.

2. OBJETIVO(S):

2.1 Objetivo General

Desarrollar diagramas UML mediante el uso de PlantUML, aplicando su sintaxis y herramientas de generación automática, para representar la estructura y el comportamiento del sistema NovaMarket.

2.2 Objetivos Específicos

- Configurar el entorno de trabajo necesario para la utilización de PlantUML en Visual Studio Code.
- Aplicar la sintaxis básica de PlantUML en la elaboración de distintos tipos de diagramas UML.
- Elaborar los diagramas de casos de uso, clases, secuencia, componentes y despliegue del sistema NovaMarket a partir de su Product Backlog.
- Validar que los diagramas generados representen correctamente los elementos y relaciones del sistema modelado.

3. MARCO TEÓRICO:

3.1 UML y su clasificación de diagramas

UML organiza sus diagramas en dos grandes categorías. Los diagramas estructurales, como el de clases, componentes y despliegue, describen los elementos que conforman el sistema y las relaciones estáticas entre ellos. Los diagramas de comportamiento, como el de casos de uso y el de secuencia, representan la manera en que dichos elementos interactúan a lo largo del tiempo o frente a distintos actores (Fowler, 2004). La Tabla 1 resume los diagramas utilizados en esta práctica y su propósito dentro del proceso de modelado.

Tabla 1. Diagramas UML desarrollados en la práctica

Diagrama	Tipo	Propósito
Casos de uso	Comportamiento	Representar los actores del sistema y sus interacciones principales.
Clases	Estructural	Definir las entidades del sistema, sus atributos, métodos y relaciones.
Secuencia	Comportamiento	Mostrar el orden temporal de los mensajes entre objetos para un caso de uso.
Componentes	Estructural	Describir los módulos del sistema y sus dependencias.

Nota: Clasificación adaptada de Fowler (2004)

3.2 Diagrama de Casos de Uso

El diagrama de casos de uso representa las funcionalidades que el sistema debe proporcionar desde la perspectiva de sus usuarios. Mediante este diagrama se identifican los actores externos que interactúan con la aplicación y las acciones que

cada uno puede realizar. Por esta razón, resulta útil para expresar los requisitos funcionales y verificar que las funciones consideradas durante el análisis correspondan con las necesidades planteadas por los usuarios (Ambler, 2005, pp. 33–46).

Los principales elementos de un caso de uso son:

• Actor. Quién o qué interactúa con el sistema, aquellos elementos externos a nuestro sistema

• Caso de Uso. Una funcionalidad del sistema.

• Límite de un sistema. Es un rectángulo que engloba los casos de uso de nuestra aplicación.

• Asociación de Comunicación. Representa la comunicación entre los actores y los casos de uso

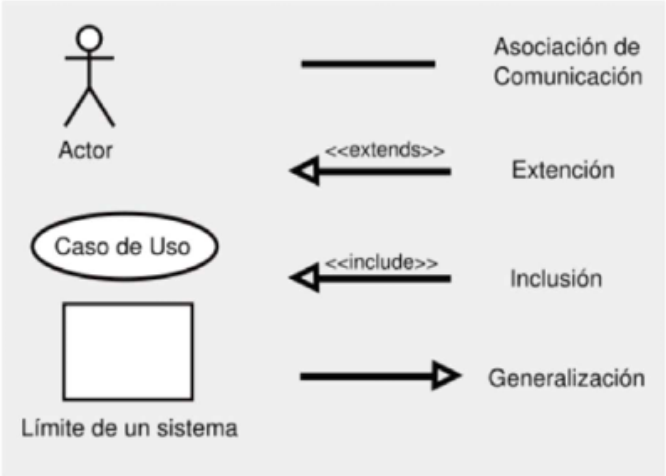
• Extensión. Relación de dependencia entre casos de uso, indica cuando un caso de uso es una especialización de otro.

• Inclusión. Relación de dependencia entre casos de uso que indica cuando un caso de uso emplea a otro.

• Generalización. Representa jerarquías entre los actores. Es decir, cuando unos actores son una especialización de otros.

Como se puede evidenciar en la Figura 1, se muestran los diferentes elementos que componen al Diagrama de Casos de Uso y su forma de representarlos gráficamente.

Figura 1: Elementos de un Diagrama de Casos de Uso



Nota: Elaboración propia.

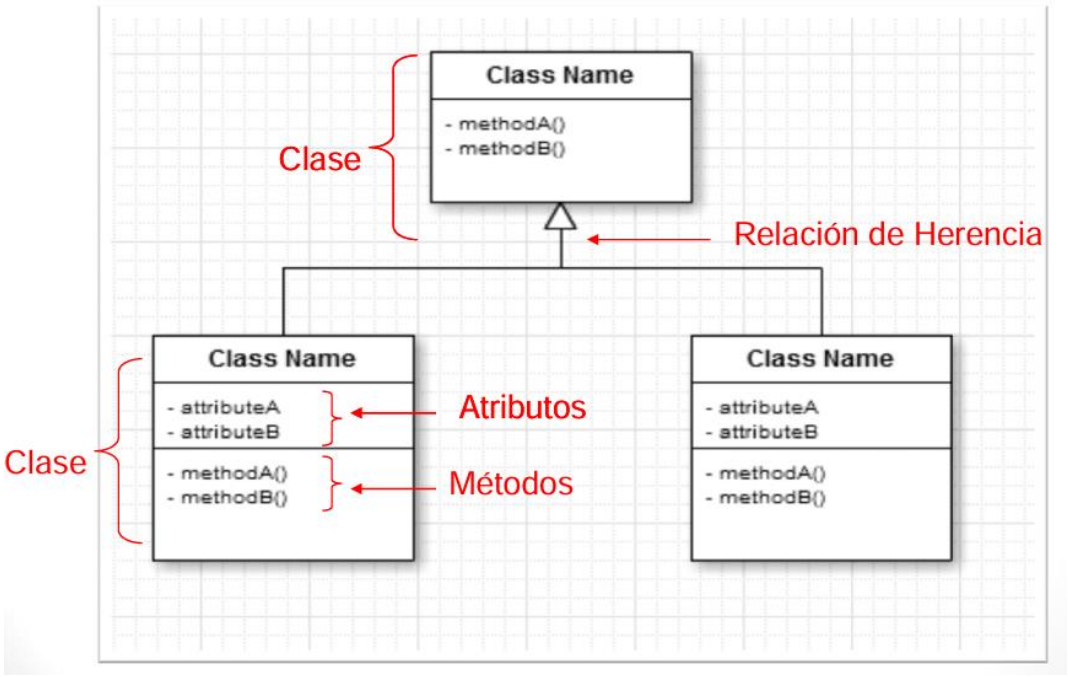
3.3 Diagrama de Clases

El diagrama de clases representa la estructura estática de un sistema mediante sus clases, atributos, operaciones y relaciones. Una clase puede entenderse como una representación general de un conjunto de objetos que comparten características y comportamientos. Los atributos contienen los datos o propiedades de la clase, mientras que las operaciones o métodos representan las acciones que sus objetos pueden ejecutar.

Este tipo de diagrama puede utilizarse para identificar conceptos del dominio, analizar los requisitos estructurales y diseñar detalladamente un sistema orientado a objetos. También permite representar relaciones como asociación, dependencia, herencia, agregación y composición. En conjunto, el modelo de clases proporciona una descripción de las entidades que integran el sistema y de la forma en que se vinculan entre ellas (Ambler, 2005, pp. 47–72).

Como se puede observar en la Figura 2, se muestra la estructura de un Diagrama de Clases con los elementos que lo componen.

Figura 2: Elementos del Diagrama de Clases



Nota: Elaboración propia.

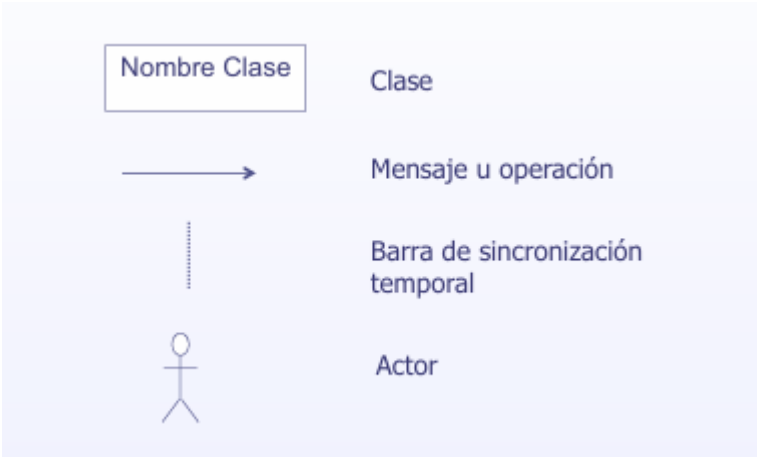
3.4 Diagrama de Secuencia

El diagrama de secuencia es una técnica de modelado dinámico que representa la interacción entre los participantes de un sistema durante la ejecución de un proceso. Los participantes se muestran mediante líneas de vida y la comunicación se representa a través de mensajes organizados siguiendo el orden en que ocurren.

Este diagrama permite comprobar la lógica y la integridad de un escenario de uso, estudiar la colaboración entre los objetos y observar la invocación de las operaciones definidas en las clases. Asimismo, puede ayudar a identificar objetos con una cantidad elevada de responsabilidades, procesos complejos y posibles problemas en la distribución de las tareas. Por ello, se utiliza para describir detalladamente cómo se realiza una funcionalidad previamente identificada en un caso de uso (Ambler, 2005, pp. 80–93).

En la Figura 3, se observa la notación para realizar un Diagrama de Secuencia de manera correcta.

Figura 3: Notación Diagrama de Secuencia



Nota: Elaboración propia.

3.5 Diagrama de Componentes

El diagrama de componentes representa la organización modular del software y las dependencias existentes entre sus partes. Un componente constituye una unidad funcional del sistema que puede proporcionar o requerir interfaces para comunicarse con otros componentes. Estos elementos pueden representar módulos, servicios, bibliotecas, subsistemas, archivos ejecutables o partes independientes de una aplicación.

Además de los componentes, el diagrama puede incluir clases de implementación y artefactos físicos, como archivos de código fuente, archivos binarios, ejecutables, scripts y tablas. Su utilización permite describir la configuración técnica del software, identificar dependencias y comprender la manera en que se integran los diferentes módulos que forman la solución informática (Ambler, 2005, pp. 132–138).

3.6 Diagrama de Despliegue

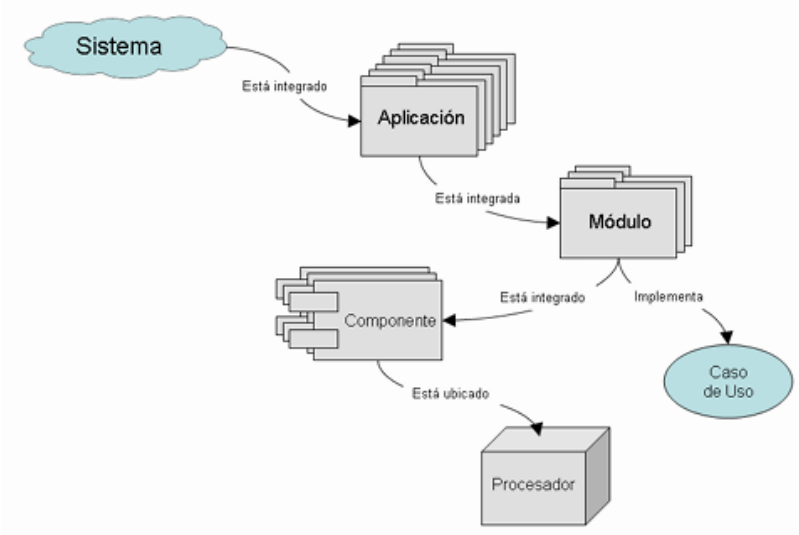
El diagrama de despliegue muestra la configuración física y de ejecución de un sistema. Su propósito es representar los nodos de hardware o los entornos de ejecución, los componentes o artefactos instalados en ellos y las conexiones mediante las cuales se comunican.

Los nodos pueden representar servidores, computadoras personales, dispositivos móviles, equipos de red o entornos virtuales. Los artefactos corresponden a elementos de software desplegables, como aplicaciones, servicios, archivos ejecutables o bases de datos. Este diagrama también puede incluir el middleware utilizado para conectar los diferentes dispositivos.

Su elaboración permite estudiar la instalación del sistema en un entorno de producción, identificar dependencias con otros sistemas, diseñar la configuración del hardware y representar la topología física de la infraestructura tecnológica. Por tanto, ofrece una visión de dónde se ejecutan los elementos del software y cómo se comunican los dispositivos que los contienen (Ambler, 2005, pp. 139–147).

Como se puede observar en la Figura 4, se representan las diferentes heramientas que componen al Diagrama de Despliegue.

Figura 4: Herramientas del Diagrama de Despliegue



Nota: Elaboración propia.

3.2 PlantUML como herramienta de modelado textual

PlantUML es un lenguaje de dominio específico que permite describir diagramas UML mediante bloques de texto delimitados por las etiquetas @startuml y @enduml. Internamente, la herramienta traduce dicha descripción a un lenguaje de grafos que es procesado por el motor Graphviz para generar la representación visual del diagrama en formatos como PNG o SVG (PlantUML, s.f.). Esta característica permite integrar los diagramas dentro de repositorios de control de versiones, ya que el

archivo fuente es texto plano y sus cambios pueden compararse línea por línea, algo que no es posible con los formatos binarios producidos por los editores gráficos tradicionales.

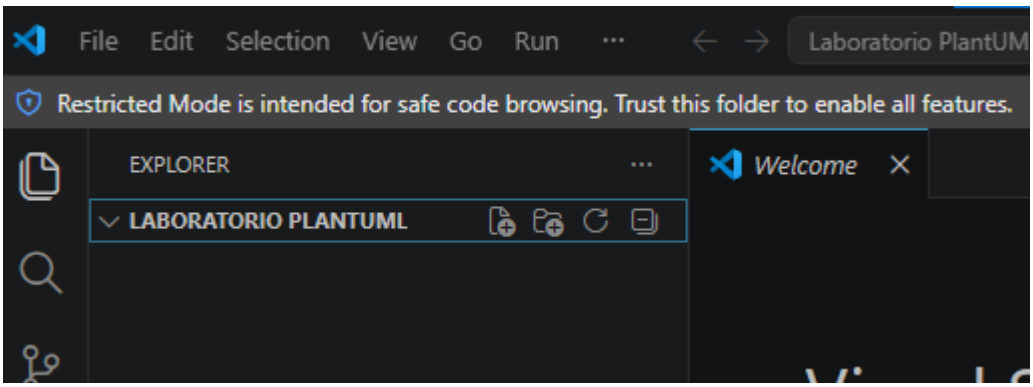
Para su funcionamiento, PlantUML requiere un entorno con Java instalado, así como Graphviz cuando se generan diagramas estructurales que dependen de la ubicación automática de nodos, como los diagramas de clases o de componentes. En Visual Studio Code, la extensión desarrollada por jebbs añade soporte de vista previa y exportación directa desde el editor, lo que agiliza el flujo de trabajo del desarrollador durante el proceso de análisis y diseño (Pressman y Maxim, 2020).

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

PASO 1 — Selección del entorno de trabajo

Se seleccionó Visual Studio Code como entorno de trabajo y se creó la carpeta Laboratorio_PlantUML como espacio de trabajo del proyecto como se muestra en la Figura 5.

Figura 5. Carpeta de trabajo Laboratorio_PlantUML abierta en Visual Studio Code.

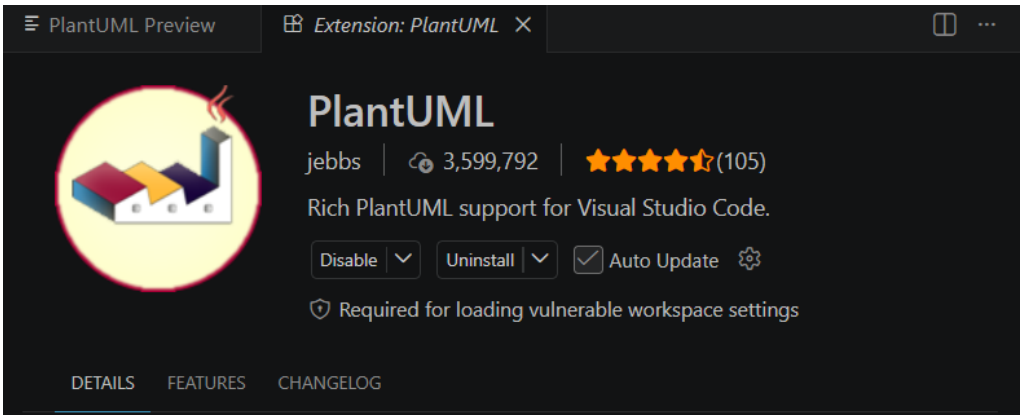


Nota: Elaboración Propia

PASO 2 — Instalación de la extensión PlantUML

Se instaló la extensión PlantUML, desarrollada por jebbs, desde el panel de extensiones de Visual Studio Code, junto con el entorno de ejecución de Java necesario para su funcionamiento como se muestra en la Figura 6.

Figura 6. Extensión PlantUML instalada en el panel de extensiones de Visual Studio Code.



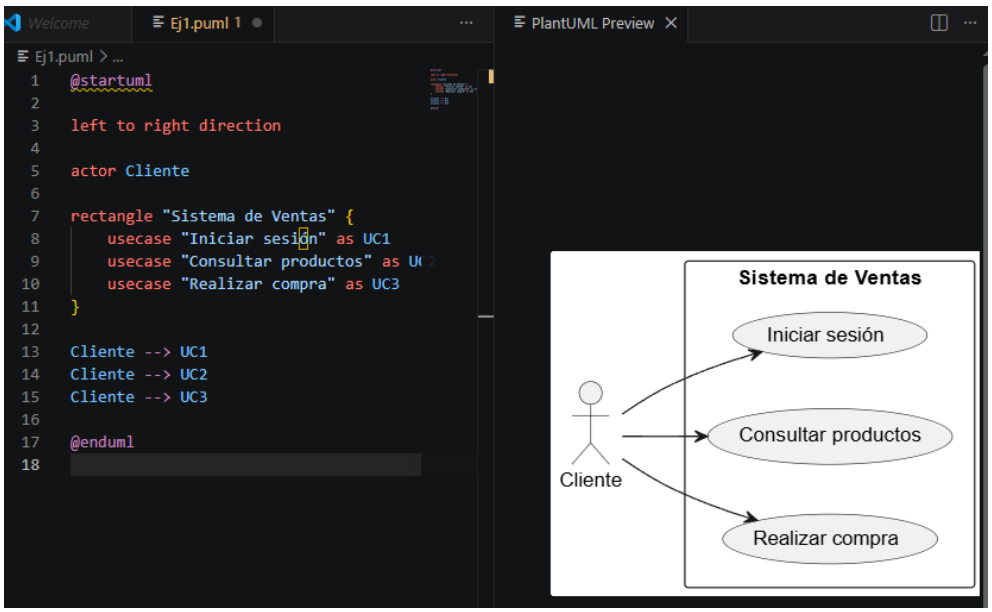
Nota: Elaboración Propia

PASO 3 y 4 — Creación y prueba de los ejemplos de Diagramas en PlantUML

A continuación, cree los archivos correspondientes y ejecute los siguientes ejemplos de diagramas UML utilizando la sintaxis de PlantUML como se muestra en la Figura 7.

Ejemplo 1. Diagrama de Casos de Uso.

Figura 7. Vista previa del diagrama de Casos de Uso de prueba en Visual Studio Code.

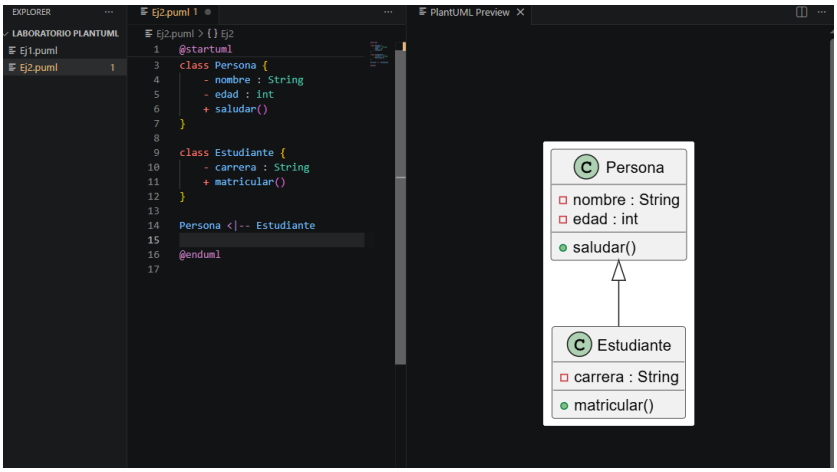


Nota: Elaboración Propia

Ejemplo 2. Diagrama de Clases

A continuación, se ejecutó un ejemplo del diagrama de clases utilizando la sintaxis de PlantUML como se muestra en la Figura 8.

Figura 8. Vista previa del diagrama de Clases de prueba en Visual Studio Code.

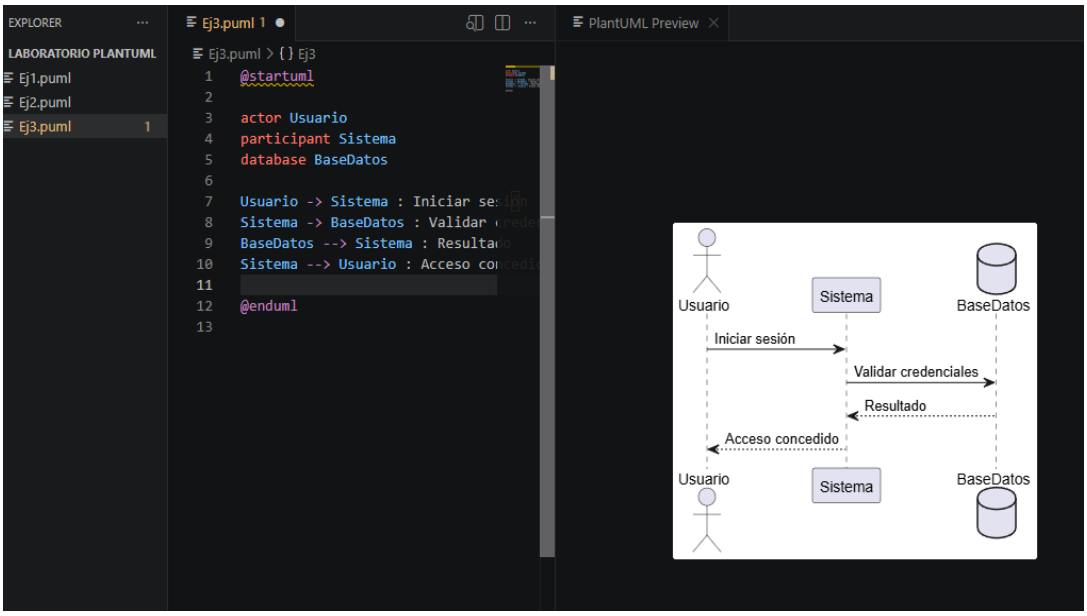


Nota: Elaboración Propia

Ejemplo 3. Diagrama de Secuencia

A continuación, se ejecutó un ejemplo del diagrama de Secuencia utilizando la sintaxis de PlantUML como se muestra en la Figura 9.

Figura 9. Vista previa del diagrama de Secuencia de prueba en Visual Studio Code.

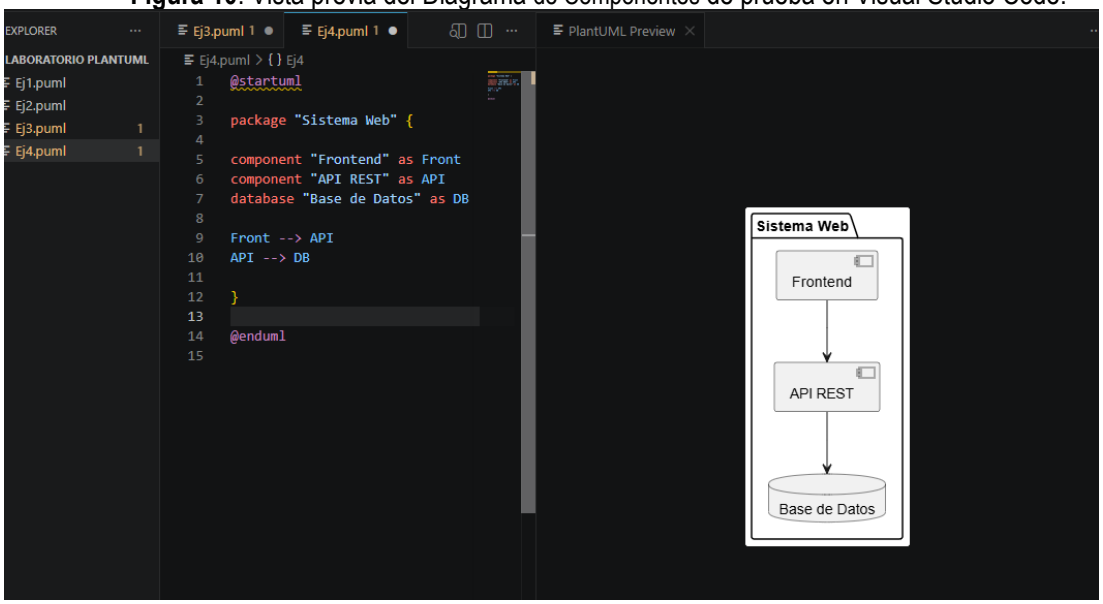


Nota: Elaboración Propia

Ejemplo 4. Diagrama de Componentes

A continuación, se ejecutó un ejemplo del diagrama de Componentes utilizando la sintaxis de PlantUML como se muestra en la Figura 10.

Figura 10. Vista previa del Diagrama de Componentes de prueba en Visual Studio Code.

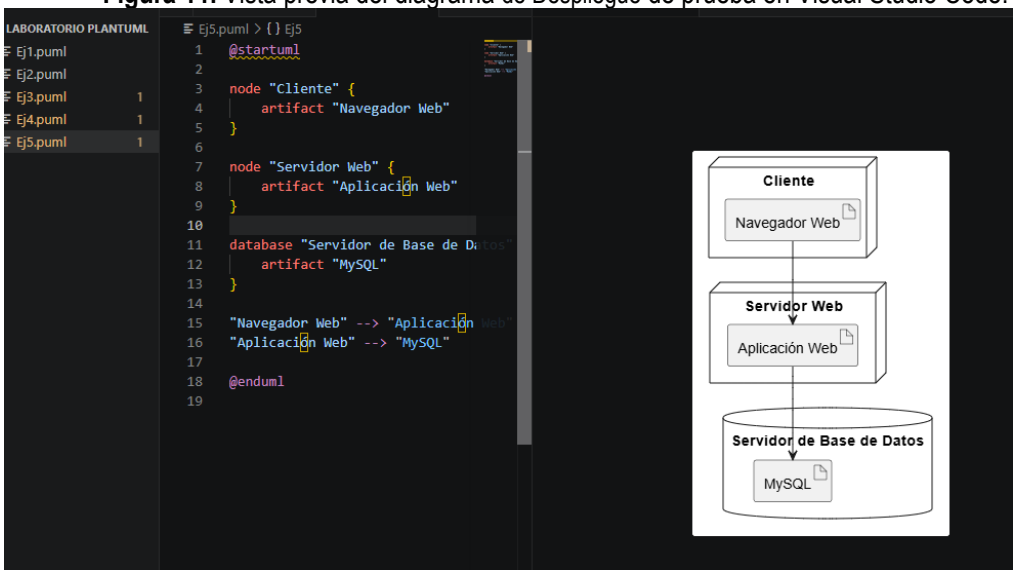


Nota: Elaboración Propia

Ejemplo 5. Diagrama de Despliegue

A continuación, se ejecutó un ejemplo del diagrama de Despliege utilizando la sintaxis de PlantUML como se muestra en la Figura 11.

Figura 11. Vista previa del diagrama de Despliegue de prueba en Visual Studio Code.



Nota: Elaboración Propia

PASO 5 — Elaboración de los diagramas del sistema NovaMarket

4.1 Diagrama de Casos de Uso

El diagrama identifica a los actores Usuario, Administrador y Cajero, correspondientes a los roles definidos en las historias de usuario HU-01 a HU-10 del Product Backlog, y describe las funcionalidades principales que cada uno puede ejecutar dentro del sistema, tal como se detalla en el código de la Figura 12.

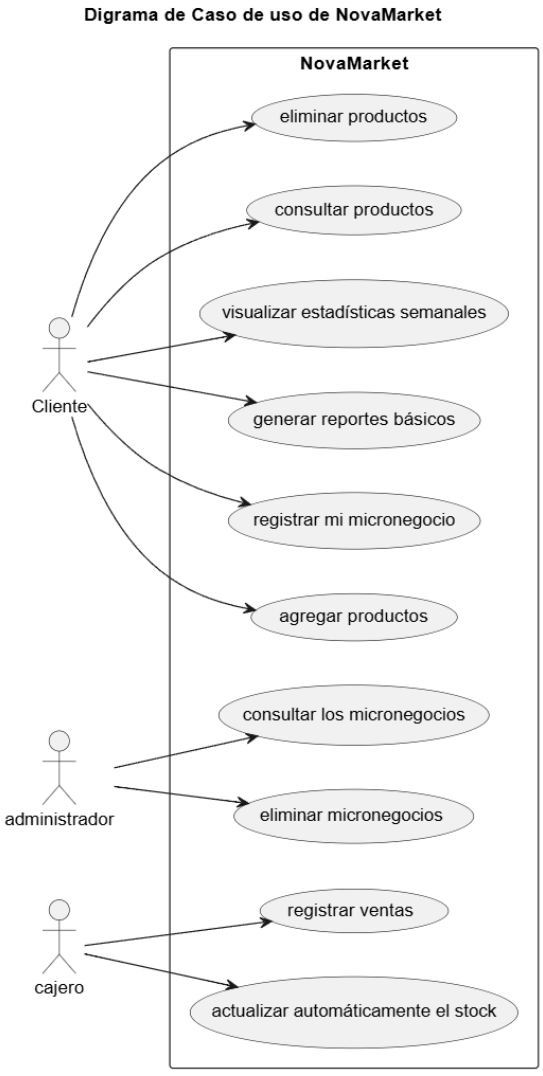
```
@startuml
title "Digrama de Caso de uso de NovaMarket"
left to right direction

actor Cliente
actor administrador
actor cajero

rectangle "NovaMarket"{
   (registrar mi micronegocio)
   (eliminar micronegocios )
   (consultar los micronegocios)
   (agregar productos)
   (eliminar productos)
   (consultar productos)
   (registrar ventas)
   (actualizar automáticamente el stock)
   (visualizar estadísticas semanales)
   (generar reportes básicos)
}
Cliente --> (registrar mi micronegocio)
```

```
Cliente --> (agregar productos)
Cliente -->  (eliminar productos)
Cliente -->  (consultar productos)
Cliente --> (visualizar estadísticas semanales)
Cliente -->  (generar reportes básicos)
administrador -->(eliminar micronegocios )
administrador -->(consultar los micronegocios)
cajero -->(registrar ventas)
cajero --> (actualizar automáticamente el stock)
@enduml
```

Figura 12. Diagrama de casos de uso del sistema NovaMarket



Nota: Elaboración Propia.

4.2 Diagrama de Clases

El diagrama de clases estructura las entidades Usuario, Administrador, Cajero, Micronegocio, Producto, Venta y Reporte, estableciendo relaciones de herencia entre Usuario y sus especializaciones, y de asociación

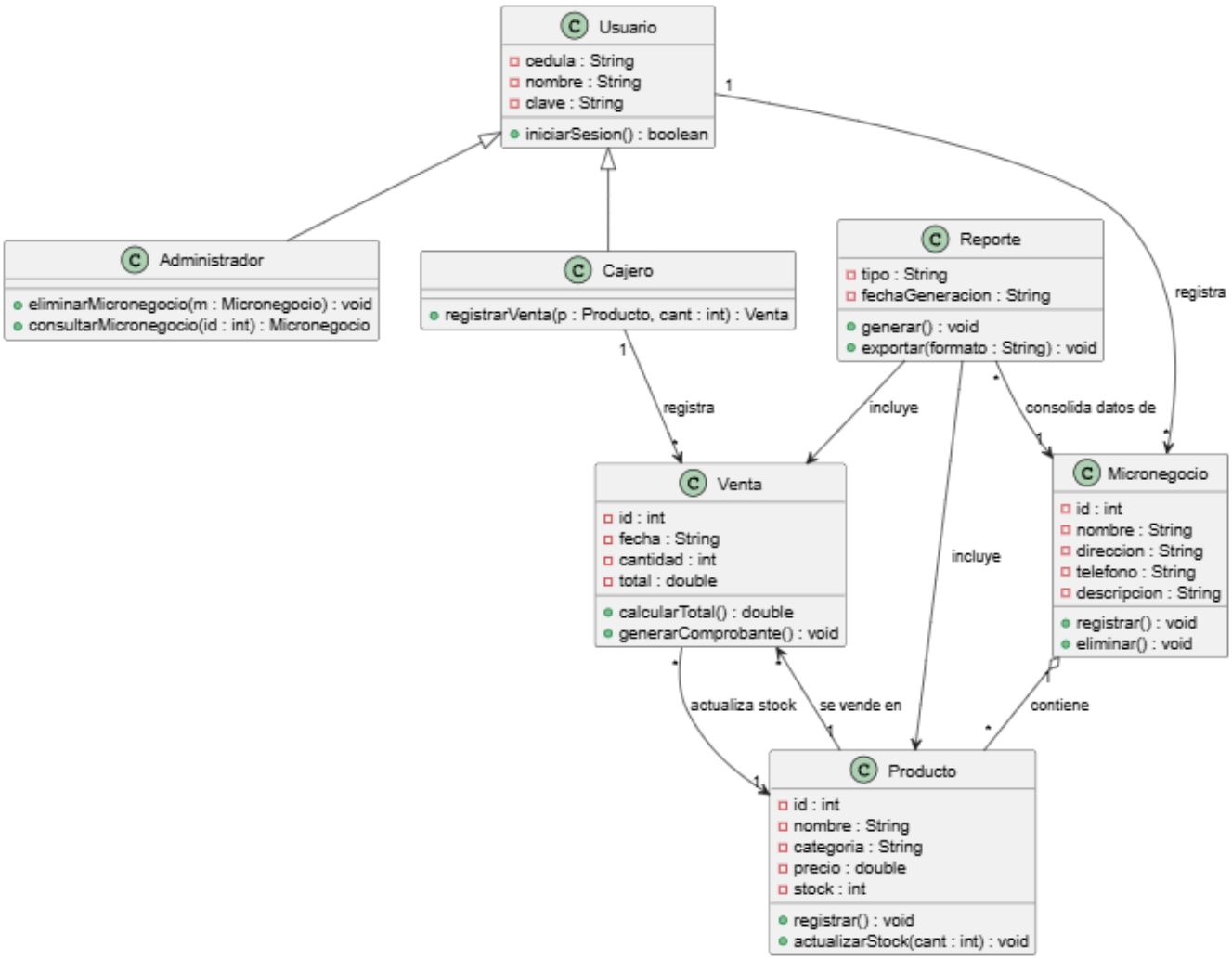
y agregación entre las clases que gestionan los micronegocios, los productos y las ventas, como se observa en la Figura 13.

Codigo Diagrama de Clases

```
@startuml
class Usuario {
  - cedula : String
  - nombre : String
  - clave : String
  + iniciarSesion() : boolean
}
class Administrador {
  + eliminarMicronegocio(m : Micronegocio) : void
  + consultarMicronegocio(id : int) : Micronegocio
}
class Cajero {
  + registrarVenta(p : Producto, cant : int) : Venta
}
class Micronegocio {
  - id : int
  - nombre : String
  - direccion : String
  - telefono : String
  - descripcion : String
  + registrar() : void
  + eliminar() : void
}
class Producto {
  - id : int
  - nombre : String
  - categoria : String
  - precio : double
  - stock : int
  + registrar() : void
  + actualizarStock(cant : int) : void
}
class Venta {
  - id : int
  - fecha : String
  - cantidad : int
  - total : double
  + calcularTotal() : double
  + generarComprobante() : void
}
class Reporte {
  - tipo : String
  - fechaGeneracion : String
  + generar() : void
  + exportar(formato : String) : void
}
Usuario <|-- Administrador
```

```
Usuario <|-- Cajero
Usuario "1" --> "*" Micronegocio : registra
Micronegocio "1" o-- "*" Producto : contiene
Producto "1" --> "*" Venta : se vende en
Cajero "1" --> "*" Venta : registra
Venta "*" --> "1" Producto : actualiza stock
Reporte "*" --> "1" Micronegocio : consolida datos de
Reporte --> Venta : incluye
Reporte --> Producto : incluye
@enduml
```

Figura 13. Diagrama de clases del sistema NovaMarket



Nota: Elaboración Propia

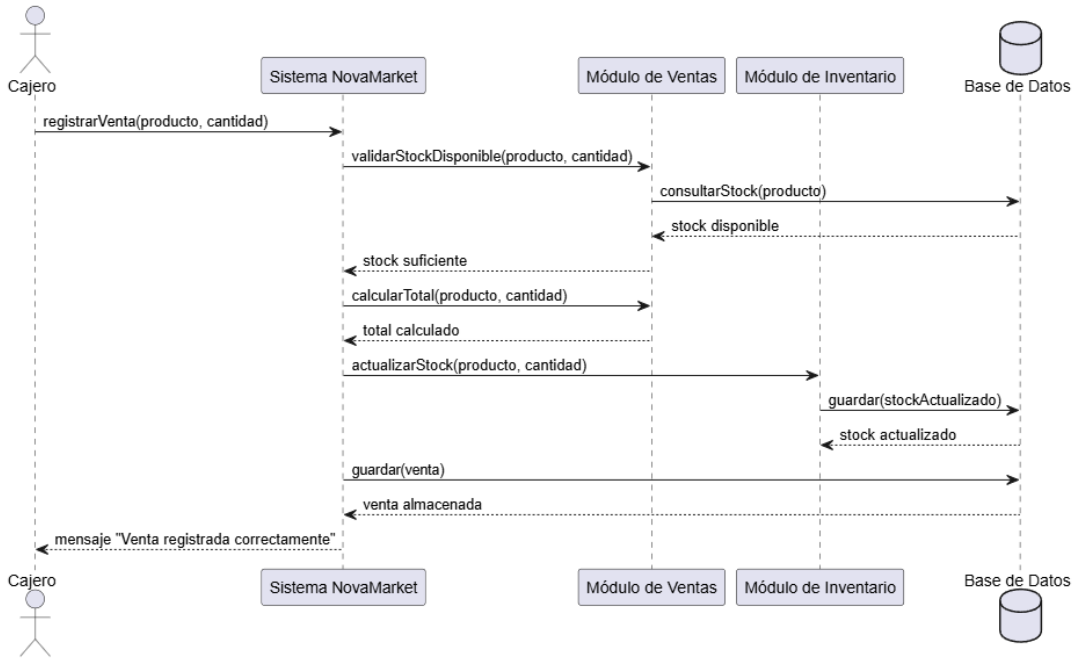
4.3 Diagrama de Secuencia

El diagrama de secuencia representa el flujo temporal del proceso de registro de una venta, desde la validación del stock disponible hasta la actualización automática del inventario, según lo descrito en la historia de usuario HU-08 e ilustrado en la Figura 14.

Codigo Diagrama de Secuencia

```
@startuml
actor Cajero
participant "Sistema NovaMarket" as Sistema
participant "Módulo de Ventas" as Ventas
participant "Módulo de Inventario" as Inventario
database "Base de Datos" as BD
Cajero -> Sistema : registrarVenta(producto, cantidad)
Sistema -> Ventas : validarStockDisponible(producto, cantidad)
Ventas -> BD : consultarStock(producto)
BD --> Ventas : stock disponible
Ventas --> Sistema : stock suficiente
Sistema -> Ventas : calcularTotal(producto, cantidad)
Ventas --> Sistema : total calculado
Sistema -> Inventario : actualizarStock(producto, cantidad)
Inventario -> BD : guardar(stockActualizado)
BD --> Inventario : stock actualizado
Sistema -> BD : guardar(venta)
BD --> Sistema : venta almacenada
Sistema --> Cajero : mensaje "Venta registrada correctamente"
@enduml
```

Figura 14. Diagrama de Secuencia del sistema NovaMarket



Nota: Elaboración Propia

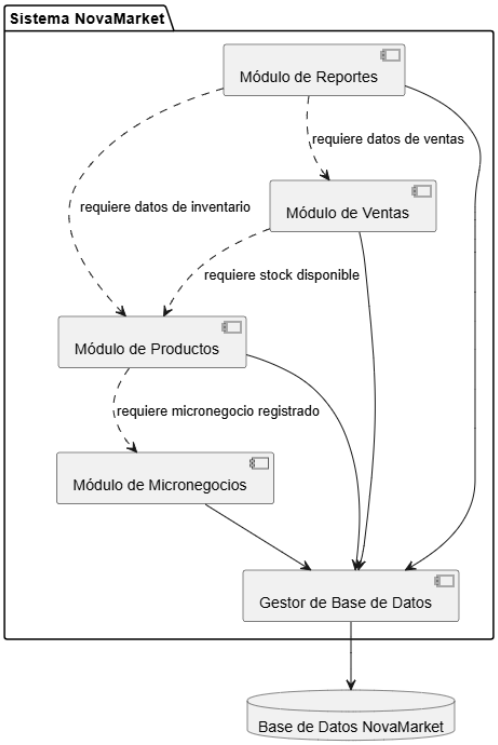
4.4 Diagrama de Componentes

El diagrama de componentes agrupa la funcionalidad del sistema en cuatro módulos alineados con las épicas del Product Backlog, gestión de micronegocios, gestión de productos, gestión de ventas y gestión de reportes, los cuales dependen de un gestor de base de datos común, tal como se muestra en la Figura 15.

Codigo Diagrama de Componentes

```
@startuml
package "Sistema NovaMarket" {
  [Módulo de Micronegocios] as Micro
  [Módulo de Productos] as Prod
  [Módulo de Ventas] as Ventas
  [Módulo de Reportes] as Reportes
  [Gestor de Base de Datos] as Gestor
}
database "Base de Datos NovaMarket" as BD
Micro --> Gestor
Prod --> Gestor
Ventas --> Gestor
Reportes --> Gestor
Gestor --> BD
Ventas ..> Prod : requiere stock disponible
Prod ..> Micro : requiere micronegocio registrado
Reportes ..> Ventas : requiere datos de ventas
Reportes ..> Prod : requiere datos de inventario
@enduml
```

Figura 15. Diagrama de Componentes del sistema NovaMarket



Nota: Elaboración propia.

4.5 Diagrama de Despliegue

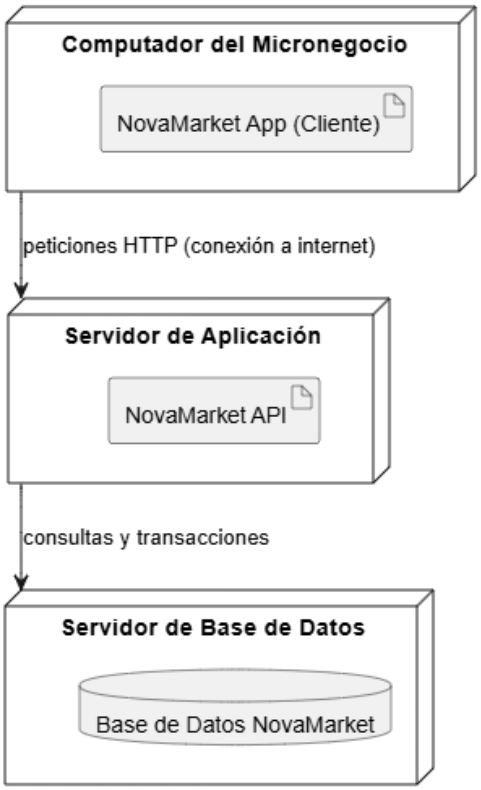
El diagrama de despliegue representa la infraestructura de ejecución del sistema, compuesta por el equipo del micronegocio que aloja la aplicación cliente, un servidor de aplicación que expone la API de NovaMarket

y un servidor de base de datos donde se almacena la información de micronegocios, productos y ventas, como se observa en la Figura 16.

Codigo Diagrama de Despliege

```
@startuml
node "Computador del Micronegocio" as PC {
  artifact "NovaMarket App (Cliente)" as App
}
node "Servidor de Aplicación" as Serv {
  artifact "NovaMarket API" as API
}
node "Servidor de Base de Datos" as DBServer {
  database "Base de Datos NovaMarket" as DB
}
PC --> Serv : peticiones HTTP (conexión a internet)
Serv --> DBServer : consultas y transacciones
@enduml
```

Figura 16. Diagrama de Secuencia del sistema NovaMarket



Nota: Elaboración propia.

4. ANÁLISIS DE RESULTADOS:

A partir de la elaboración de los cinco diagramas se obtuvo una representación completa del sistema NovaMarket desde distintas perspectivas de modelado, coherente con las épicas E-01 a E-04 definidas en el Product Backlog. La Tabla 2 resume los elementos identificados en cada diagrama y su relación con el resto del modelo.

Tabla 2. Elementos identificados por diagrama

Diagrama	Elementos principales	Relación con otros diagramas
Casos de uso	3 actores, 10 casos de uso, 1 relación include	Los casos de uso registrar venta y actualizar inventario se detallan operacionalmente en el diagrama de secuencia.
Clases	7 clases, 2 relaciones de herencia, 6 relaciones de asociación o agregación	Las clases Producto y Venta son las mismas participantes del diagrama de secuencia.
Secuencia	4 líneas de vida, 12 mensajes	Describe operacionalmente el caso de uso registrar venta identificado en el diagrama de casos de uso.
Componentes	5 componentes, 1 base de datos	Los componentes agrupan los métodos definidos en las clases del diagrama de clases.
Despliegue	3 nodos, 2 artefactos de aplicación, 1 base de datos	Representa el entorno físico donde se ejecutan los componentes identificados previamente.

El diagrama de casos de uso evidenció que la mayor parte de la funcionalidad operativa del negocio, como el registro de micronegocios, productos y la generación de reportes, recae sobre el actor Usuario, mientras que el rol de Administrador se limita a las operaciones de consulta y eliminación de micronegocios, lo cual concuerda con la asignación de historias de usuario observada en el Product Backlog de la práctica anterior. El diagrama de secuencia permitió verificar que la actualización del inventario ocurre inmediatamente después de calcular el total de una venta, evitando así que se registre una venta sin descontar el stock correspondiente, condición que en el sistema real se implementa mediante el método actualizarStock de la clase Producto.

El diagrama de componentes confirmó que los cuatro módulos funcionales dependen de un único gestor de base de datos, lo cual concuerda con la arquitectura cliente-servidor descrita en el diagrama de despliegue, donde la aplicación instalada en el micronegocio se comunica con un servidor de aplicación y este, a su vez, con un servidor de base de datos independiente.

5.1 Relación entre los diagramas elaborados

El diagrama de casos de uso (Figura 1) sirvió de base para la construcción del diagrama de clases, ya que cada funcionalidad identificada para los actores Usuario, Administrador y Cajero se tradujo directamente en un método dentro de la clase correspondiente. Por ejemplo, el caso de uso registrar micronegocio, asignado al actor Usuario, dio origen al método registrar() de la clase Micronegocio, y el caso de uso eliminar micronegocio, asignado al Administrador, se convirtió en el método eliminarMicronegocio() de dicha clase. De esta manera, el diagrama de casos de uso no solo documentó el comportamiento externo del sistema, sino que funcionó como insumo directo para definir la responsabilidad de cada clase.

A partir del diagrama de clases (Figura 2) se identificaron los objetos que debían participar en el diagrama de secuencia. Dado que el caso de uso registrar venta incluye a actualizar inventario, se tomó ese flujo como el más representativo para modelar en la Figura 3, y las clases Venta, Producto y Cajero definidas previamente permitieron establecer con precisión qué mensajes debía enviar cada participante: Sistema, Módulo de Ventas, Módulo de Inventario y Base de Datos.

El diagrama de secuencia, a su vez, evidenció la existencia de módulos funcionales diferenciados para la validación de stock, el cálculo del total, la actualización del inventario y la persistencia de la información, lo que sirvió de base para agrupar esas responsabilidades en el diagrama de componentes (Figura 4). Cada participante del diagrama de secuencia terminó representado como un componente independiente, y las dependencias detectadas en el orden de los mensajes, como el hecho de que el módulo de Ventas necesita al módulo de Productos antes de confirmar una venta, se tradujeron en las relaciones de dependencia entre componentes.

Finalmente, el diagrama de componentes permitió determinar los nodos físicos necesarios para el diagrama de despliegue (Figura 5). Como el Gestor de Base de Datos concentra el acceso a la persistencia, se decidió ubicarlo en un servidor de base de datos independiente del servidor de aplicación, mientras que los módulos restantes se agruparon en la aplicación cliente instalada en el equipo del micronegocio. En conjunto, los cinco diagramas se construyeron de manera progresiva, tomando siempre como insumo los elementos definidos en el diagrama anterior.

5. DISCUSIÓN:

Un aspecto relevante que surgió durante la práctica fue la facilidad con la que PlantUML permitió mantener la coherencia terminológica entre los distintos diagramas. Al reutilizar los mismos nombres de clases, atributos y métodos definidos en el diagrama de clases dentro del diagrama de secuencia y del diagrama de componentes, se evitaron ambigüedades que suelen presentarse cuando distintos integrantes de un equipo elaboran cada diagrama de forma independiente en herramientas gráficas separadas. Esta consistencia terminológica resultó particularmente útil al momento de redactar el análisis de resultados, ya que permitió rastrear un mismo elemento, como el método actualizarStock de la clase Producto, a través de tres diagramas distintos sin necesidad de reinterpretarlo cada vez. Esto sugiere que, más allá de la generación gráfica automática, uno de los mayores beneficios de PlantUML radica en imponer una disciplina de nomenclatura que favorece la trazabilidad del modelo completo, un aspecto que rara vez se discute en la literatura centrada únicamente en la notación UML.

Además, la práctica permitió reconocer una limitación propia del enfoque textual: la disposición automática de los elementos dentro del diagrama, calculada por Graphviz, no siempre coincide con la organización visual más clara para el lector. En diagramas con múltiples relaciones, como el de clases o el de componentes, algunas líneas de asociación se cruzaron entre sí, lo que exigió reordenar manualmente el orden de declaración de las clases dentro del código para mejorar la legibilidad del resultado final. Esta limitación no invalida las ventajas señaladas anteriormente, pero sí evidencia que el modelado textual requiere cierta experiencia previa para anticipar cómo el motor de generación ubicará los elementos, y que la revisión visual del diagrama exportado sigue siendo un paso necesario antes de incorporarlo a la documentación definitiva del proyecto.

6. CONCLUSIONES:

La comparación entre los diagramas elaborados y el Product Backlog de la práctica de artefactos Scrum permitió verificar que la funcionalidad planificada para NovaMarket se mantuvo coherente a lo largo de las distintas técnicas de documentación utilizadas en el curso. Los roles de Usuario, Administrador y Cajero, definidos originalmente en las historias de usuario, se conservaron sin alteraciones en el diagrama de casos de uso y en el diagrama de clases, lo que confirma que el paso de un enfoque ágil basado en historias de usuario hacia un modelado formal en UML no implicó pérdida de información ni redefinición de responsabilidades. Esta continuidad entre metodologías refuerza la utilidad de UML como complemento del backlog ágil, y no como una alternativa que lo reemplace.

El proceso de modelado también dejó en evidencia decisiones de diseño que aún no habían sido discutidas por el equipo, como la arquitectura cliente-servidor asumida en el diagrama de despliegue. Durante la práctica de artefactos Scrum, el Product Backlog se centró en describir la funcionalidad del sistema desde la perspectiva del usuario, sin especificar en qué tipo de infraestructura se ejecutaría NovaMarket ni cómo se comunicarían sus distintos componentes. Fue recién al construir el diagrama de despliegue cuando el equipo tuvo que definir explícitamente que la aplicación cliente instalada en el micronegocio se conectaría a un servidor de aplicación y este, a su vez, a un servidor de base de datos independiente. Esto permite concluir

que la elaboración de diagramas UML no es únicamente una tarea de documentación posterior al desarrollo, sino una herramienta que anticipa decisiones técnicas necesarias antes de iniciar la implementación, obligando al equipo a resolver preguntas de arquitectura que de otro modo quedarían pendientes hasta etapas más avanzadas del proyecto. Abordar estas decisiones en la etapa de modelado, en lugar de postergarlas hasta la fase de codificación, reduce el riesgo de reestructurar el sistema una vez que el desarrollo ya esté en marcha, y permite planificar con mayor anticipación aspectos como el hospedaje del servidor o la escalabilidad del sistema para múltiples micronegocios.

Finalmente, la práctica evidenció que la calidad de un diagrama UML depende tanto de la herramienta utilizada como de la claridad conceptual previa del sistema que se pretende modelar. PlantUML resolvió eficazmente la parte gráfica y de renderizado, generando diagramas ordenados y legibles a partir de unas pocas líneas de código, pero la correcta definición de actores, clases y relaciones dependió directamente de haber trabajado antes con las épicas y las historias de usuario de NovaMarket durante la práctica de artefactos Scrum. Cuando esa base conceptual no estaba clara, como ocurrió inicialmente con el alcance exacto del rol de Cajero frente al de Usuario, el diagrama generado por la herramienta reflejaba esa ambigüedad en lugar de resolverla, lo que obligó a regresar al Product Backlog para precisar la responsabilidad de cada actor antes de continuar con el modelado. Esta situación permite concluir que ninguna herramienta de modelado, por automatizada que sea, sustituye el análisis previo del problema; su verdadero valor está en traducir ese análisis en una representación gráfica precisa, versionable y fácil de mantener a lo largo del ciclo de vida del proyecto, siempre que el equipo llegue a la etapa de diagramación con las reglas de negocio del sistema suficientemente definidas.

7. BIBLIOGRAFÍA:

Booch, G., Rumbaugh, J., y Jacobson, I. (2005). *The unified modeling language user guide* (2.a ed.). Addison-Wesley.

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.a ed.). Addison-Wesley.

PlantUML. (s.f.). *PlantUML language reference guide*. Recuperado el 19 de julio de 2026, de https://plantuml.com

Pressman, R. S., y Maxim, B. R. (2020). *Ingeniería del software: Un enfoque práctico* (9.a ed.). McGraw-Hill.

Roques, P. (2009). *UML 2 par la pratique: Cours et exercices*. Eyrolles.